

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
----------	-----------	--------------

p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

```
if (x > y || p < q)
```

Intermediate code:

```
if x > y goto L1
if p < q goto L1
goto L2
```

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control

Given:

22
Aug
26

CSA1407-compiler
CO4-AT3 Design

Balaji K.
192324255

① Intermediate code Analysis and Optimization

Equivalent exp tree:

Root : - left (l), right (r)

Subtree : left (*), right (w)

Subtree * : left +, right z)

Subtree + : left x, right y)

Expression : ((x + y) * z) / w) - p

Role : (temp var) $\rightarrow$ break complex multi operator expressions into simple 3 address instructions, making target machine code generation straight forward.

② Symbol table and memory Allocation

Total Row Size:

$$1 + 8 + 4 + 4 = 17 \text{ Bytes.}$$

Memory Allocation with Standard Alignment

char p : 1 byte

double q : 8 byte

int r : 4 byte

float s : 4 bytes

total Aligned memory : 24 Bytes.

Impact of Alignment:

Data type requires address alignment to Optimize CPU memory fetches (which introduces padding bytes and increases memory footprint to prevent unaligned access penalties).

Q3. Optimization in Intermediate Code

Step-by-step trace:

$$1) t_1 = m * n$$

$$2) t_2 = p + q$$

$$3) t_3 = m * n$$

$$4) t_4 = t_2 - t_3$$

Common sub Expression Elimination:

Replace t_3 with existing result t_1 .

Optimized code:

$$t_1 = m * n$$

$$t_2 = p + q$$

$$t_4 = t_2 - t_1$$

Execution Efficiency: Reduces clock cycles and instruction count by eliminating repeated arithmetic operations.

④ Control flow and Boolean Expression Handling:

Control flow Interpretation:

Evaluate $x > y$; if true, branch immediately to L_1 (Then).
if false evaluate $p < q$: if true, branch to L_n . if both fail, proceed to L_2 (else-clause)

Short-Circuit Evaluation:

If $x > y$ is true, the OR condition is guaranteed to be true, so $p < q$ is skipped entirely, minimizing runtime instruction execution.

⑤ Backpatching and Flow Control.

→ Role of Backpatching

Fills in target label addresses for incomplete jump instructions in a single pass during syntax-directed translation eliminating the need for a separate pass.

Demonstration:

→ true list = {200} →

Instruction 200 updated to:

200: if $x > y$ goto L-true 1

→ false list = {201} →

Instruction 201 updated to:

201: goto L-false 1

Importance:

Resolves forward jumps in Boolean and control flow statements clearly when target destination labels are not known at time of code generation.

Instruction Target

200: if x==y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand